

用户使用手册（补）

本文档是基于AxVisor技术文档补全的用户使用手册内容，包含1.8非常规过程、1.9操作命令一览表、1.10程序文件和数据文件一览表、1.11用户操作举例四个部分。

1.8 非常规过程

本章节提供应急或非常规操作的必要信息及操作步骤，包括出错处理操作、向后备系统切换操作以及维护人员须知的操作和注意事项。

1.8.1 应急操作

1.8.1.1 系统启动失败应急处理

应急情况： AxVisor系统无法正常启动

应急步骤：

- 1. 检查硬件虚拟化支持
 - 确认CPU支持硬件虚拟化扩展（Intel VT-x或ARM VE）
 - 检查BIOS/UEFI设置中硬件虚拟化是否已启用
- 2. 串口调试分析
 - 连接串口调试线，波特率设置为115200
 - 观察启动日志，确定具体错误位置
 - 检查内存区域初始化是否成功
- 3. 配置文件检查
 - 验证VM配置文件TOML语法是否正确
 - 确认镜像文件路径和加载地址配置是否有效
 - 检查设备树文件是否存在且格式正确

恢复方法：

```
# 重新编译AxVisor镜像
./axvisor.sh build

# 检查配置文件语法
toml-validate configs/vms/linux-aarch64-*.toml

# 重新生成设备树
dtc -I dts -O dtb -o configs/vms/linux-aarch64-rk3568_smp1.dtb configs/vms/linux-aarch64-rk3568_smp1.dts
```

1.8.1.2 虚拟机运行异常应急处理

应急情况： 虚拟机启动后异常退出或无响应

应急步骤：

1. 检查虚拟机状态
- 观察虚拟机退出代码和错误信息

◦ 确认vCPU分配是否与配置一致

◦ 检查内存分配是否充足
2. 资源配置验证
- 验证物理CPU集合配置是否正确

◦ 检查内存区域映射是否有冲突

◦ 确认设备直通配置是否有效
3. 客户机镜像检查
- 验证客户机内核镜像完整性

◦ 检查设备树与虚拟机配置的匹配性

◦ 确认ramdisk或rootfs是否可用

恢复方法：

```
# 重启单个虚拟机（如果支持）
# 通过串口进入AxVisor调试模式
# 检查虚拟机配置并重新加载
```

1.8.2 故障诊断程序

1.8.2.1 系统级故障诊断

诊断流程：

1. 硬件层面检查
- CPU虚拟化扩展支持检测

◦ 内存容量和完整性测试

◦ 串口通信功能验证
2. 软件层面检查
- AxVisor镜像完整性验证

◦ 配置文件语法和逻辑检查

◦ 依赖库和工具链版本确认
3. 运行时环境检查
- 启动日志详细分析

◦ 内存分配和映射状态检查

◦ 中断处理和设备状态监控

诊断工具：

- 串口调试终端
- JTAG调试器（如可用）
- 主机端交叉编译工具链
- 文件系统挂载工具

1.8.2.2 虚拟机级故障诊断

诊断要点：

1. 虚拟机配置诊断

- VM ID唯一性检查
- CPU数量与物理CPU集合匹配性
- 内存区域重叠检测
- 设备直通冲突分析

2. 镜像加载诊断

- 内核镜像格式验证
- 加载地址有效性检查
- DTB设备树兼容性验证
- 文件系统访问权限确认

3. 运行时状态诊断

- vCPU执行状态监控
- 内存访问异常检测
- 设备I/O操作追踪
- 系统调用和超调用分析

1.8.3 向后备系统切换

1.8.3.1 Type-1模式到Type-1.5模式切换

切换场景：当Type-1裸机模式出现严重故障时，切换到宿主Linux环境运行

切换步骤：

1. 准备宿主Linux环境

```
# 确保宿主Linux支持模块加载
sudo modprobe kvm-intel # 或 kvm-amd

# 检查虚拟化支持
lscpu | grep Virtualization
```

2. 编译Type-1.5模式AxVisor

```
# 修改配置以支持Linux主机
./axvisor.sh defconfig
# 编辑.hvconfig.toml设置适当的平台配置
./axvisor.sh build
```

3. 部署和启动

```
# 加载AxVisor内核模块
sudo insmod axvisor.ko

# 使用axcli工具管理虚拟机
sudo ./axcli enable
sudo ./axcli vm create --name backup_vm --cfg-path backup_config.toml
```

1.8.3.2 主虚拟机到备用虚拟机切换

切换触发条件：

- 主虚拟机出现不可恢复错误
- 资源耗尽或性能严重下降
- 安全威胁检测

切换程序：

1. 状态保存（如果可能）
 - 保存虚拟机运行状态
 - 备份关键数据和配置
2. 备用系统激活
 - 启动预配置的备用虚拟机
 - 恢复必要的服务和数据
 - 重新配置网络和设备连接

1.8.4 维护人员须知

1.8.4.1 日常维护注意事项

安全操作规程：

1. 系统更新维护
 - 定期备份虚拟机配置文件
 - 保持工具链和依赖库最新版本
 - 测试环境验证后再更新生产环境
2. 硬件维护

- 定期检查串口连接稳定性
- 监控系统温度和功耗
- 确保存储设备健康状态

3. 软件维护

- 监控系统日志和错误报告
- 定期清理临时文件和日志
- 保持配置文件版本管理

1.8.4.2 紧急联系和技术支持

技术支持渠道：

- AxVisor社区：<https://github.com/arceos-hypervisor/axvisor>
- 技术文档：<https://arceos-hypervisor.github.io/axvisorbook/>
- 问题反馈：通过GitHub Issues提交问题报告

紧急操作权限：

- 具备串口调试权限的技术人员
- 熟悉硬件平台的维护工程师
- 具有系统管理员权限的运维人员

1.9 操作命令一览表

本章节按字母顺序逐个列出全部操作命令的格式、功能及参数说明。

1.9.1 构建相关命令

axvisor.sh

功能： AxVisor项目主要构建和配置脚本

格式： `./axvisor.sh [command] [options]`

命令参数：

- `defconfig` - 生成默认配置文件.hvconfig.toml
- `build` - 编译AxVisor镜像
- `clean` - 清理构建产生的临时文件
- `help` - 显示帮助信息

使用示例：

```
./axvisor.sh defconfig
./axvisor.sh build
./axvisor.sh clean
```

bootm

功能： U-Boot引导命令，用于启动AxVisor镜像

格式： `bootm <kernel_addr> [initrd_addr] <dtb_addr>`

参数说明：

- `kernel_addr` - AxVisor内核镜像在内存中的地址
- `initrd_addr` - 初始RAM盘地址（可选，用"-"表示无）
- `dtb_addr` - 设备树二进制文件在内存中的地址

使用示例：

```
bootm 0x4000000 - 0x10000000
```

build.sh

功能： 客户机Linux镜像构建脚本（用于ROC-RK3568-PC SDK）

格式： `./build.sh [target] [options]`

参数说明：

- 无参数 - 交互式选择配置并构建
- `cleanall` - 清理所有构建输出并重新选择配置

使用示例：

```
./build.sh  
./build.sh cleanall
```

1.9.2 调试相关命令

dtc

功能： 设备树编译器，用于DTS和DTB之间的转换

格式： `dtc [options] -I <input_format> -O <output_format> -o <output_file> <input_file>`

参数说明：

- `-I` - 输入格式（dts或dtb）
- `-O` - 输出格式（dtb或dts）
- `-o` - 输出文件名
- `input_file` - 输入文件名

使用示例：

```
dtc -I dts -O dtb -o linux-aarch64-rk3568_smp1.dtb linux-aarch64-rk3568_smp1.dts
dtc -I dtb -O dts -o output.dts input.dtb
```

genimage

功能： 生成磁盘镜像文件的工具

格式： `genimage --inputpath <input_dir> --outputpath <output_dir> --config <config_file> --tmppath <tmp_dir> --rootpath <root_dir>`

参数说明：

- `--inputpath` - 输入文件目录
- `--outputpath` - 输出文件目录
- `--config` - 配置文件路径
- `--tmppath` - 临时文件目录
- `--rootpath` - 根文件系统目录

使用示例：

```
genimage --inputpath ./input --outputpath ./output --config genimage-sd.cfg --tmppath ./tmp --rootpath ./root
```

git

功能： 版本控制系统，用于获取源代码

格式： `git [command] [options] [arguments]`

常用命令：

- `clone` - 克隆远程仓库
- `checkout` - 切换分支
- `pull` - 拉取更新
- `status` - 查看状态

使用示例：

```
git clone --recurse-submodules https://github.com/arceos-hypervisor/axvisor.git
git checkout rk3588_jd4
git pull origin master
```

1.9.3 文件操作命令

insmod

功能： Linux内核模块加载命令（Type-1.5模式）

格式： `insmod <module_file>`

参数说明：

- `module_file` - 内核模块文件路径（.ko文件）

使用示例：

```
sudo insmod /path/to/axvisor.ko
```

load

功能： U-Boot文件加载命令

格式： `load <interface> <device:partition> <addr> <filename>`

参数说明：

- `interface` - 存储接口类型（mmc、usb等）
- `device:partition` - 设备号和分区号
- `addr` - 加载到内存的地址
- `filename` - 要加载的文件名

使用示例：

```
load mmc 0:8 0x4000000 /boot/Image_root_aarch64_rk3588
load mmc 0:8 0x10000000 /boot/rk3588.bin
```

make

功能： 编译构建工具

格式： `make [target] [variables]`

常用目标和变量：

- `ARCH=aarch64` - 指定目标架构
- `PLATFORM=<platform_config>` - 指定平台配置
- `SMP=<num>` - 指定SMP核心数量
- `VM_CONFIGS=<config_files>` - 指定虚拟机配置文件
- `LOG=<level>` - 指定日志级别

使用示例：

```
make ARCH=aarch64 PLATFORM=configs/platforms/aarch64-rk3588j-hv.toml SMP=4
make A=examples/helloworld PLATFORM=aarch64-dyn SMP=1 LOG=info
```


mkimage

功能： U-Boot镜像生成工具

格式： `mkimage [options] -f <its_file> <output_file>`

参数说明：

- `-f` - 指定FIT镜像描述文件（.its文件）
- `output_file` - 输出的镜像文件名

使用示例：

```
mkimage -f kernel.its fitImage  
../host/bin/mkimage_phypi -f kernel.its ./input/fitImage
```

mount / umount

功能： 文件系统挂载和卸载命令

格式： `mount <device> <mount_point> / umount <mount_point>`

参数说明：

- `device` - 设备文件或镜像文件
- `mount_point` - 挂载点目录

使用示例：

```
sudo mount boot.img boot  
sudo umount boot  
sudo mount -o loop rootfs.img /mnt/rootfs
```

1.9.4 系统管理命令

repo

功能： Google Repo工具，用于管理多Git仓库项目

格式： `repo <command> [options]`

常用命令：

- `sync` - 同步所有仓库
- `start` - 开始一个新的分支

使用示例：

```
.repo/repo/repo sync -l
.repo/repo/repo start firefly --all
.repo/repo/repo sync -c --no-tags
```

rustc / cargo

功能： Rust编程语言编译器和包管理工具

格式： rustc [options] <input_file> / cargo [command] [options]

常用命令：

- rustc --version - 查看Rust版本
- cargo build - 构建项目
- cargo clean - 清理构建产物

使用示例：

```
rustc --version
cargo build --release
```

sudo

功能： 以超级用户权限执行命令

格式： sudo <command> [arguments]

使用示例：

```
sudo insmod axvisor.ko
sudo mount boot.img boot
sudo ./axcli enable
```

1.10 程序文件和数据文件一览表

本章节按文件名字母顺序或按功能与模块分类顺序逐个列出文件名称、标识符及说明。

1.10.1 AxVisor核心文件

1.10.1.1 可执行镜像文件

文件名	文件类型	功能说明	生成方式
axvisor_aarch64-generic.bin	二进制镜像	AxVisor AArch64架构核心镜像文件	./axvisor.sh build
axvisor_aarch64-dyn.bin	二进制镜像	AxVisor动态平台镜像文件	make编译生成

文件名	文件类型	功能说明	生成方式
Image_root_aarch64_rk3588	二进制镜像	适用于RK3588平台的AxVisor镜像	交叉编译生成
axvisor.ko	内核模块	Type-1.5模式下的Linux内核模块	内核模块编译

1.10.1.2 构建脚本文件

文件名	文件类型	功能说明	位置路径
axvisor.sh	Shell脚本	AxVisor主要构建和配置脚本	项目根目录
build.rs	Rust脚本	编译时配置处理脚本	src/build.rs
Makefile	Make文件	项目构建规则定义	项目根目录
Cargo.toml	TOML配置	Rust项目依赖和配置	项目根目录

1.10.2 配置文件

1.10.2.1 AxVisor配置文件

文件名	文件类型	功能说明	位置路径
.hvconfig.toml	TOML配置	AxVisor主配置文件	项目根目录
aarch64-generic.toml	TOML配置	通用AArch64平台配置	configs/platforms/
aarch64-rk3588j-hv.toml	TOML配置	RK3588平台特定配置	configs/platforms/

1.10.2.2 虚拟机配置文件

文件名	文件类型	功能说明	位置路径
linux-aarch64-rk3568_smp1.toml	TOML配置	RK3568单核Linux虚拟机配置	configs/vms/
linux-aarch64-rk3568_smp2.toml	TOML配置	RK3568双核Linux虚拟机配置	configs/vms/
arceos-aarch64-rk3568_smp1.toml	TOML配置	RK3568单核ArceOS虚拟机配置	configs/vms/
linux-aarch64-e2000_smp1.toml	TOML配置	飞腾派单核Linux虚拟机配置	configs/vms/
linux-aarch64-e2000_smp2.toml	TOML配置	飞腾派双核Linux虚拟机配置	configs/vms/
arceos-aarch64-e2000_smp1.toml	TOML配置	飞腾派单核ArceOS虚拟机配置	configs/vms/

1.10.3 设备树文件

1.10.3.1 设备树源文件

文件名	文件类型	功能说明	位置路径
linux-aarch64-rk3568_smp1.dts	DTS源文件	RK3568单核Linux设备树源码	configs/vms/

文件名	文件类型	功能说明	位置路径
linux-aarch64-rk3568_smp2.dts	DTS源文件	RK3568双核Linux设备树源码	configs/vms/
arceos-aarch64-rk3568_smp1.dts	DTS源文件	RK3568单核ArceOS设备树源码	configs/vms/
linux-aarch64-e2000_smp1.dts	DTS源文件	飞腾派单核Linux设备树源码	configs/vms/
linux-aarch64-e2000_smp2.dts	DTS源文件	飞腾派双核Linux设备树源码	configs/vms/
arceos-aarch64-e2000_smp1.dts	DTS源文件	飞腾派单核ArceOS设备树源码	configs/vms/

1.10.3.2 设备树二进制文件

文件名	文件类型	功能说明	生成方式
linux-aarch64-rk3568_smp1.dtb	DTB二进制	编译后的RK3568单核Linux设备树	dtc编译生成
linux-aarch64-rk3568_smp2.dtb	DTB二进制	编译后的RK3568双核Linux设备树	dtc编译生成
rk3588.bin	DTB二进制	RK3588平台设备树二进制文件	SDK构建生成
phytiumpi_firefly.dtb	DTB二进制	飞腾派平台设备树二进制文件	SDK构建生成

1.10.4 客户机镜像文件

1.10.4.1 Linux客户机镜像

文件名	文件类型	功能说明	位置路径
Image	二进制镜像	Linux内核镜像文件	kernel/arch/arm64/boot/
vmlinux	ELF可执行文件	带符号信息的Linux内核	kernel/
zImage	压缩镜像	压缩的Linux内核镜像	kernel/arch/arm64/boot/
Image-5.10.198	二进制镜像	特定版本的Linux内核镜像	boot分区

1.10.4.2 ArceOS客户机镜像

文件名	文件类型	功能说明	生成方式
arceos-aarch64.bin	二进制镜像	ArceOS AArch64架构镜像	make编译生成
helloworld_aarch64-dyn.bin	二进制镜像	ArceOS Hello World示例	make A=examples/helloworld

1.10.5 系统镜像文件

1.10.5.1 文件系统镜像

文件名	文件类型	功能说明	位置路径
rootfs.img	EXT4镜像	根文件系统镜像	output/firmware/

文件名	文件类型	功能说明	位置路径
rootfs.ext2	EXT2镜像	EXT2格式根文件系统镜像	构建输出目录
boot.img	EXT4镜像	启动分区镜像	output/firmware/
rk356x_ubuntu_rootfs.img	EXT4镜像	Ubuntu根文件系统镜像	prebuilt_rootfs/

1.10.5.2 引导镜像

文件名	文件类型	功能说明	位置路径
fitImage	FIT镜像	飞腾派FIT格式启动镜像	构建输出目录
sdcard.img	磁盘镜像	SD卡完整镜像文件	output/
fip-all.bin	二进制镜像	飞腾派固件+U-Boot组合	构建输出目录

1.10.6 开发工具文件

1.10.6.1 SDK工具

文件名	文件类型	功能说明	位置路径
sdk_tools.sh	Shell脚本	SDK解压和同步工具	SDK根目录
build.sh	Shell脚本	ROC-RK3568-PC构建脚本	SDK根目录
make.sh	Shell脚本	U-Boot构建脚本	u-boot/

1.10.6.2 工具链文件

文件名	文件类型	功能说明	位置路径
aarch64-linux-gnu-*	可执行文件	AArch64交叉编译工具链	/usr/bin/
mkimage	可执行文件	U-Boot镜像生成工具	host/bin/
genimage	可执行文件	磁盘镜像生成工具	host/bin/
dtc	可执行文件	设备树编译器	/usr/bin/

1.10.7 配置模板文件

1.10.7.1 构建配置模板

文件名	文件类型	功能说明	位置路径
defconfig	配置文件	默认构建配置	configs/
firefly_rk3568_roc-rk3568-pc_ubuntu_defconfig	配置文件	ROC-RK3568-PC Ubuntu配置	configs/
phytiumpi_desktop_defconfig	配置文件	飞腾派桌面版配置	configs/

1.10.7.2 镜像配置模板

文件名	文件类型	功能说明	位置路径
genimage-sd.cfg	配置文件	SD卡镜像生成配置	board/phytium/
kernel.its	ITS文件	fitImage描述文件	package/phyuboot/src/

1.10.8 日志和调试文件

1.10.8.1 系统日志文件

文件名	文件类型	功能说明	生成方式
axvisor.log	文本日志	AxVisor运行日志	串口输出重定向
build.log	文本日志	构建过程日志	构建过程生成
boot.log	文本日志	启动过程日志	串口输出记录

1.10.8.2 调试信息文件

文件名	文件类型	功能说明	生成方式
vm_dump.txt	文本文件	虚拟机状态转储	调试时生成
memory_map.txt	文本文件	内存布局信息	调试输出
device_tree_dump.dts	DTS文件	设备树反汇编输出	dtc反汇编

1.11 用户操作举例

本章节提供具体的用户操作示例，展示AxVisor在不同场景下的完整操作流程。

1.11.1 在QEMU上运行AxVisor + ArceOS客户机

1.11.1.1 环境准备

操作目的： 在QEMU虚拟化环境中快速验证AxVisor功能

前置条件：

- Ubuntu 20.04或更高版本Linux系统
- QEMU已安装并支持AArch64仿真
- Rust工具链已正确安装

操作步骤：

1. 获取AxVisor源代码

```
# 克隆AxVisor仓库
git clone --recurse-submodules https://github.com/arceos-
```

```
hypervisor/axvisor.git
cd axvisor

# 检查Rust环境
rustc --version
# 确保输出类似: rustc 1.70.0-nightly
```

2. 准备ArceOS客户机镜像

```
# 克隆ArceOS源代码
git clone https://github.com/arceos-hypervisor/arceos -b hypervisor
cd arceos

# 编译ArceOS Hello World示例
make A=examples/helloworld PLATFORM=aarch64-dyn SMP=1 LOG=info

# 验证生成的镜像文件
ls -la target/aarch64-dyn/release/examples/
# 应该看到 helloworld_aarch64-dyn.bin 文件
```

3. 配置AxVisor

```
cd ../axvisor

# 生成默认配置
./axvisor.sh defconfig

# 编辑配置文件
nano .hvconfig.toml
```

配置文件内容示例：

```
# Platform for Axvisor
plat = "aarch64-generic"
# Build arguments for ArceOS
arceos_args = ["BUS=mmio", "LOG=debug"]
# ArceOS additional features
arceos_features = []
vmconfigs = ["configs/vms/arceos-aarch64.toml"]
```

4. 构建AxVisor镜像

```
# 编译AxVisor
./axvisor.sh build
```

```
# 验证生成的镜像
ls -la target/aarch64-generic/release/
# 应该看到 axvisor_aarch64-generic.bin 文件
```

1.11.1.2 启动和运行

1. 使用QEMU启动AxVisor

```
# 创建启动脚本
cat > run_qemu.sh << 'EOF'
#!/bin/bash
qemu-system-aarch64 \
    -machine virt,gic-version=2 \
    -cpu cortex-a57 \
    -smp 2 \
    -m 1G \
    -kernel target/aarch64-generic/release/axvisor_aarch64-generic.bin \
    -nographic \
    -serial mon:stdio
EOF

chmod +x run_qemu.sh
```

2. 执行启动

```
./run_qemu.sh
```

3. 预期输出

```
d8888                                     .d88888b.  .d8888b.
      d88888                                     d88P" "Y88b d88P  Y88b
      d88P888                                     888      888 Y88b.
      d88P 888 888d888 .d8888b. .d88b. 888      888 "Y888b.
      d88P 888 888P" d88P"    d8P  Y8b 888      888 "Y88b.
      d88P 888 888      888      888888888 888      888 "888
      d88888888888 888      Y88b.    Y8b.    Y88b. .d88P Y88b  d88P
```

d88P 888 888 "Y8888P "Y8888 "Y88888P" "Y8888P"

arch = aarch64 platform = aarch64-qemu-virt-hv target = aarch64-unknown-none-softfloat build_mode = release log_level = info smp = 1

[0.020822 0 axruntime:130] Logging is enabled. [0.026419 0 axruntime:131] Primary CPU 0 started, dtb = 0x44000000. [0.028520 0 axruntime:133] Found physcial memory regions: ... [0.295531 0:2 axvisor::vmm::config:33] Creating VM [1] "arceos" ... Hello, world!

1.11.1.3 结果验证

****成功标志：****

1. AxVisor成功启动并显示启动信息
2. 虚拟机"arceos"被成功创建
3. ArceOS客户机启动并输出"Hello, world!"
4. 系统正常关机

1.11.2 在ROC-RK3568-PC开发板上运行AxVisor + Linux客户机

1.11.2.1 硬件准备

****操作目的：**** 在真实硬件平台上部署AxVisor并运行Linux客户机

****硬件要求：****

- ROC-RK3568-PC开发板
- USB转TTL串口调试线
- SD卡或eMMC存储设备
- 网络连接（可选）

****软件环境：****

- Ubuntu 24.04开发主机
- ROC-RK3568-PC官方SDK
- 串口调试终端（如minicom或picocom）

1.11.2.2 SDK环境搭建

1. ****下载并解压SDK****

```
```bash
```

```
从官方网站下载SDK压缩包
```

```
假设下载文件为 ROC-RK3568-PC_SDK_xxx.tar.gz
```

```
解压SDK
```

```
chmod +x ./sdk_tools.sh
```

```
mkdir ../firefly_rk3568_sdk
```

```
./sdk_tools.sh --unpack -C ../firefly_rk3568_sdk
```

```
恢复SDK内容
```

```
cd ../firefly_rk3568_sdk
```

```
.repo/repo/repo sync -l
```

```
.repo/repo/repo start firefly --all
```

## 2. 修复Python兼容性问题（如果使用Python 3.12+）

```
修改main.py
```

```
sed -i 's/import imp/import importlib.util as imp/' .repo/repo/main.py
```

```
创建formatter.py文件
```

```
（此处省略具体内容，参考技术文档中的完整代码）
```

```
cat > .repo/repo/formatter.py << 'EOF'
完整的formatter.py内容
EOF
```

### 3. 安装构建依赖

```
sudo apt install git ssh make gcc libssl-dev liblz4-tool expect expect-dev \
 g++ patchelf chrpath gawk texinfo chrpath diffstat binfmt-
support \
 qemu-user-static live-build bison flex fakeroot cmake gcc-
multilib \
 g++-multilib unzip device-tree-compiler ncurses-dev
libgucharmap-2-90-dev \
 bzip2 expat cpp-aarch64-linux-gnu libgmp-dev libmpc-dev bc
python-is-python3
```

#### 1.11.2.3 构建Linux客户机镜像

##### 1. 配置并构建SDK

```
执行构建脚本
./build.sh

选择配置文件
输入 30 选择: firefly_rk3568_roc-rk3568-pc_ubuntu_defconfig
30

等待构建完成（可能需要几小时）
构建完成后检查输出文件
ls -la output/
```

##### 2. 验证生成的文件

```
ls -la output/firmware/
应该看到：
boot.img - 启动分区镜像
rootfs.img - 根文件系统镜像
其他相关文件
```

#### 1.11.2.4 构建AxVisor镜像

##### 1. 准备虚拟机配置

```
cd /path/to/axvisor

编译设备树
dtc -I dts -O dtb -o configs/vms/linux-aarch64-rk3568_smp1.dtb \
 configs/vms/linux-aarch64-rk3568_smp1.dts

检查虚拟机配置文件
cat configs/vms/linux-aarch64-rk3568_smp1.toml
```

## 2. 配置AxVisor构建

```
生成配置文件
./axvisor.sh defconfig

编辑配置
nano .hvconfig.toml
```

配置示例：

```
plat = "aarch64-generic"
arceos_args = ["BUS=mmio", "LOG=debug"]
arceos_features = []
vmconfigs = ["configs/vms/linux-aarch64-rk3568_smp1.toml"]
```

## 3. 构建AxVisor

```
构建
./axvisor.sh build

验证输出
ls -la target/aarch64-generic/release/axvisor_aarch64-generic.bin
```

### 1.11.2.5 镜像部署

#### 1. 修改boot.img

```
cd /path/to/firefly_rk3568_sdk

挂载boot镜像
mkdir boot_mount
sudo mount output/firmware/boot.img boot_mount

替换内核镜像
sudo cp /path/to/axvisor/target/aarch64-generic/release/axvisor_aarch64-
```

```
generic.bin \
 boot_mount/Image-5.10.198

卸载
sudo umount boot_mount
```

## 2. 修改rootfs.img（从文件系统加载模式）

```
挂载根文件系统
mkdir rootfs_mount
sudo mount output/firmware/rootfs.img rootfs_mount

创建客户机文件目录
sudo mkdir -p rootfs_mount/guest

复制客户机镜像和设备树
sudo cp kernel/arch/arm64/boot/Image rootfs_mount/guest/
sudo cp /path/to/axvisor/configs/vms/linux-aarch64-rk3568_smp1.dtb \
 rootfs_mount/guest/

卸载
sudo umount rootfs_mount
```

### 1.11.2.6 硬件部署和启动

#### 1. 烧写镜像

```
使用瑞芯微官方烧写工具或dd命令
将修改后的boot.img和rootfs.img烧写到对应分区

如果使用SD卡部署：
sudo dd if=output/firmware/boot.img of=/dev/sdX1 bs=1M
sudo dd if=output/firmware/rootfs.img of=/dev/sdX2 bs=1M
sync
```

#### 2. 连接串口调试

```
连接USB转TTL线到开发板串口
设置串口参数：115200波特率，8数据位，无奇偶校验，1停止位

使用picocom连接
picocom -b 115200 /dev/ttyUSB0

或使用minicom
minicom -D /dev/ttyUSB0 -b 115200
```

### 3. 启动开发板

```
上电启动开发板
在U-Boot阶段观察启动日志
如果需要手动引导，在U-Boot提示符下执行：

从存储设备加载镜像
load mmc 0:8 0x4000000 /boot/Image-5.10.198
load mmc 0:8 0x10000000 /boot/rk3588.bin

启动AxVisor
bootm 0x4000000 - 0x10000000
```

#### 1.11.2.7 运行验证

##### 预期启动日志：

```
d8888 .d88888b. .d8888b.
 d88888 d88P" "Y88b d88P Y88b
 d88P888 888 888 Y88b.
 d88P 888 888d888 .d8888b .d88b. 888 888 "Y888b.
 d88P 888 888P" d88P" d8P Y8b 888 888 "Y88b.
 d88P 888 888 888 88888888 888 888 "888
d88888888888 888 Y88b. Y8b. Y88b. .d88P Y88b d88P
d88P 888 888 "Y888P "Y8888 "Y88888P" "Y8888P"

arch = aarch64
platform = aarch64-rk3588-hv
target = aarch64-unknown-none-softfloat
build_mode = release
log_level = info
smp = 4

[0.020822 0 axruntime:130] Logging is enabled.
[0.026419 0 axruntime:131] Primary CPU 0 started, dtb = 0x44000000.
...
[0.295531 0:2 axvisor::vmm::config:33] Creating VM [1] "linux"
...
[1.234567 0:5 axvisor::vmm::vcpus:243] VM[1] Vcpu[0] running...

Linux客户机启动
Linux version 5.10.198
...
Welcome to Ubuntu 20.04.3 LTS!
login:
```

##### 验证步骤：

1. 确认AxVisor成功启动并显示logo

2. 检查虚拟机创建日志，确认"Creating VM [1] "linux""
3. 观察Linux客户机启动过程
4. 等待Linux登录提示符出现
5. 使用默认用户名和密码登录（参考SDK文档）

#### 常见问题排查：

- 如果无法启动，检查串口连接和波特率设置
- 如果没有虚拟机启动，检查客户机镜像和设备树路径
- 如果出现内存错误，检查虚拟机配置文件中的内存分配

### 1.11.3 在飞腾派开发板上运行AxVisor + 多虚拟机

#### 1.11.3.1 环境准备

**操作目的：** 在飞腾派平台上同时运行Linux和ArceOS两个客户机

#### 硬件要求：

- 飞腾派开发板（E2000处理器）
- SD卡（至少32GB）
- USB转TTL串口线
- 以太网线（可选）

#### 1.11.3.2 SDK环境搭建

##### 1. 获取飞腾派SDK

```
克隆飞腾派OS构建系统
git clone https://gitee.com/phytium_embedded/phytium-pi-os.git
cd phytium-pi-os

切换到稳定分支（可选）
git checkout master # 或 2024.02
```

##### 2. 安装构建依赖

```
sudo apt update
sudo apt install build-essential git wget cpio unzip rsync bc libncurses5-
dev \
libssl-dev device-tree-compiler u-boot-tools flex bison \
python3-setuptools
```

##### 3. 修复Ubuntu 24.04兼容性问题

```
修改Buildroot配置以支持新版glibc
具体修复参考技术文档
```

### 1.11.3.3 构建客户机镜像

#### 1. 构建Linux客户机

```
配置并构建
make phytiumpi_desktop_defconfig
make

验证输出
ls -la output/images/
应该看到:
Image - Linux内核镜像
rootfs.ext2 - 根文件系统
fip-all.bin - 固件
```

#### 2. 构建ArceOS客户机

```
cd /path/to/arceos

编译ArceOS
make A=examples/helloworld PLATFORM=aarch64-dyn SMP=1 LOG=info

验证生成的镜像
ls -la target/aarch64-dyn/release/examples/helloworld_aarch64-dyn.bin
```

### 1.11.3.4 配置多虚拟机

#### 1. 准备虚拟机配置文件

```
cd /path/to/axvisor

编译设备树
dtc -I dts -O dtb -o configs/vms/linux-aarch64-e2000_smp1.dtb \
 configs/vms/linux-aarch64-e2000_smp1.dts
dtc -I dts -O dtb -o configs/vms/arceos-aarch64-e2000_smp1.dtb \
 configs/vms/arceos-aarch64-e2000_smp1.dts
```

#### 2. 配置AxVisor支持多虚拟机

```
生成配置
./axvisor.sh defconfig

编辑配置支持多虚拟机
nano .hvconfig.toml
```

多虚拟机配置示例：

```
plat = "aarch64-generic"
arceos_args = ["BUS=mmio", "LOG=debug"]
arceos_features = []
vmconfigs = [
 "configs/vms/linux-aarch64-e2000_smp1.toml",
 "configs/vms/arceos-aarch64-e2000_smp1.toml"
]
```

### 3. 检查虚拟机配置文件

```
检查Linux虚拟机配置
cat configs/vms/linux-aarch64-e2000_smp1.toml

检查ArceOS虚拟机配置
cat configs/vms/arceos-aarch64-e2000_smp1.toml

确保两个虚拟机的ID、CPU分配不冲突
```